Algorithmique I - Corrigé du chapitre 4

Modularisation · Syllabus 2026–2027 · Exercices 1 à 7

Les paramètres sont déclarés dans l'en-tête : `int somme(int x, y)`. Un algorithme qui retourne un entier commence par `int`, et la valeur est transmise avec `return`. Un algorithme `void` ne retourne aucune valeur. Les variables de chaque module sont locales. Conformément au cours, les solutions comportent au plus **un** `return` **final par module**.

Exercice 1 - À l'appel !

Source : p. 40 (PDF 49). Le module utilisé est `int somme(int x, y)`, qui renvoie `x+y`. Le principal lit `a=5`, `b=10`.

Ligne	Verdict	Motif
4 : `somme(a,b)`	À écarter	Dans la convention de cet exercice, le retour doit être utilisé comme expression ; appel isolé sans utilisation
5 : `a = somme(a,b)`	Correct	Deux entiers définis, affectation du retour
6 : `print somme(a,3)`	Correct	Affichage du retour
7 : `print somme(a,a)`	Correct	Arguments définis
8 : `print somme(a,c)`	Incorrect	`c` n'a pas encore été initialisé
9 : `c = somme(a,b)`	Correct	Initialise `c`
10 : `print somme(a,c)`	Correct	`c` est maintenant défini
11 : `print somme(x,y)`	Incorrect	`x` et `y` n'existent que dans le module `somme`
12 : `print somme(a,b,c)`	Incorrect	Trois arguments fournis pour deux paramètres
13 : `print somme(a,a+b)`	Correct	Une expression peut être un argument
14 : `a = a+somme(a,b)`	Correct	Retour employé dans une expression
15 : `b = somme(a,somme(a,b))`	Correct	Appels imbriqués avec arguments entiers

Traçage de `principal`, après suppression des lignes 4, 8, 11 et 12. Les `print` ne modifient pas les variables ; leurs sorties figurent dans le tableau suivant.

Ligne	a	b	c
2 - Déclarations	-	-	-
3 - `read a, b`	5	10	-
5 - `a = somme(a,b)`	15	10	-
9 - `c = somme(a,b)`	15	10	25
14 - `a = a+somme(a,b)`	40	10	25
15 - `b = somme(a,somme(a,b))`	40	90	25

Ligne de `principal`	Affichage
6	`18`
7	`30`
10	`40`
13	`40`

Traçages locaux de `somme`. Chaque appel crée ses propres paramètres `x`, `y` et une nouvelle variable `z` initialement non définie. La ligne 20 affecte `z` ; la ligne 21 retourne sa valeur sans modifier les variables du module.

Appel 1 - ligne 5 : `somme(5, 10)`.

Ligne de `somme`	x	y	z
18–19 - Paramètres et déclaration	5	10	-
20 - `z = x+y`	5	10	15

Retour (ligne 21) : `15`.

Appel 2 - ligne 6 : `somme(15, 3)`.

Ligne de `somme`	x	y	z
18–19 - Paramètres et déclaration	15	3	-
20 - `z = x+y`	15	3	18

Retour (ligne 21) : `18`.

Appel 3 - ligne 7 : `somme(15, 15)`.

Ligne de `somme`	x	y	z
18–19 - Paramètres et déclaration	15	15	-
20 - `z = x+y`	15	15	30

Retour (ligne 21) : `30`.

Appel 4 - ligne 9 : `somme(15, 10)`.

Ligne de `somme`	x	y	z
18–19 - Paramètres et déclaration	15	10	-
20 - `z = x+y`	15	10	25

Retour (ligne 21) : `25`.

Appel 5 - ligne 10 : `somme(15, 25)`.

Ligne de `somme`	x	y	z
18–19 - Paramètres et déclaration	15	25	-
20 - `z = x+y`	15	25	40

Retour (ligne 21) : `40`.

Appel 6 - ligne 13 : `somme(15, 25)`.

Ligne de `somme`	x	y	z
18–19 - Paramètres et déclaration	15	25	-
20 - `z = x+y`	15	25	40

Retour (ligne 21) : `40`.

Appel 7 - ligne 14 : `somme(15, 10)`.

Ligne de `somme`	x	y	z
18–19 - Paramètres et déclaration	15	10	-
20 - `z = x+y`	15	10	25

Retour (ligne 21) : `25`.

Appel 8 - ligne 15 - intérieur : `somme(40, 10)`**.**

Ligne de `somme`	x	y	z
18–19 - Paramètres et déclaration	40	10	-
20 - `z = x+y`	40	10	50

Retour (ligne 21) : `50`.

Appel 9 - ligne 15 - extérieur : `somme(40, 50)`**.**

Ligne de `somme`	x	y	z
18–19 - Paramètres et déclaration	40	50	-
20 - `z = x+y`	40	50	90

Retour (ligne 21) : `90`.

À la ligne 14, `a` devient `15 + 25 = 40`. À la ligne 15, l'appel intérieur retourne `50`, puis l'appel extérieur `90`. **Affichages successifs :** `18`, `30`, `40`, `40`. Un appel isolé dont le résultat est ignoré peut être syntaxiquement admis par un langage exécutable ; la ligne 4 est écartée selon la convention pédagogique retenue pour cet exercice, et non comme interdiction universelle.

Exercice 2 - On trace !

Source : p. 41 (PDF 50). `principal` lit `a=1`, `b=4`. Chaque fonction est tracée dans son propre contexte ; les paramètres locaux ne modifient pas directement les variables du principal.

Trace de `principal`**.**

Ligne	a	b
2 - Déclarations	-	-
3 - `read a, b`	1	4
4 - `a = calculs(a, 2*b)`	27	4
5 - `b = calculs(b, a)`	27	32

Affichage (ligne 6) : `27`.

Premier appel : `calculs(1, 8)` **(ligne 4 de** `principal`**).**

Ligne de `calculs`	a	b	c
9–10 - Paramètres et déclaration	1	8	-
11 - `c = a+b`	1	8	9
12 - `a = fonction(c)`	27	8	9

Retour (ligne 13) : `27`.

Appel imbriqué : `fonction(9)` **(ligne 12 de** `calculs`**).** La condition `x > 10` est fausse, donc seule la ligne 21 affecte `y`.

Ligne de `fonction`	x	y
16–17 - Paramètre et déclaration	9	-
21 - `y = 2*x`	9	18

Retour (ligne 22) : `x+y = 9+18 = 27`.

Deuxième appel : `calculs(4, 27)` **(ligne 5 de** `principal`**).**

Ligne de `calculs`	a	b	c
9–10 - Paramètres et déclaration	4	27	-
11 - `c = a+b`	4	27	31
12 - `a = fonction(c)`	32	27	31

Retour (ligne 13) : `32`.

Appel imbriqué : `fonction(31)` **(ligne 12 de** `calculs`**).** La condition `x > 10` est vraie, donc seule la ligne 19 affecte `y`.

Ligne de `fonction`	x	y
16–17 - Paramètre et déclaration	31	-
19 - `y = x MOD 10`	31	1

Retour (ligne 22) : `x+y = 31+1 = 32`.

Exercice 3 - Maximum de 3 nombres

Source : p. 41 (PDF 50). **Le programme proposé est valide** : il conserve le maximum partiel dans `a`, puis réutilise `b` pour lire le troisième nombre. Aucune correction fonctionnelle n'est nécessaire.

Programme de départ : voir le syllabus, p. 41.

Avec `5,12,7` : après le premier appel `a=12`, après le deuxième `a=12`, donc affichage `Le maximum des 3 nombres vaut 12`. On suppose disponible le module `int maximum(int x,y)` défini dans la théorie.

Exercice 4 - Réécriture (sélection explicite)

Source : p. 42 (PDF 51). L'énoncé laisse le choix des algorithmes du chapitre 3. Quatre variantes sont retenues pour couvrir les quatre types de retour et remplacer entièrement lectures et affichages par **paramètres et** `return`.

a. Maximum de trois entiers - retour `int` **(chap. 3, ex. 6).**

```
int maximum3(int a, b, c)
    int resultat
    resultat = a
    if b > resultat
        resultat = b
    if c > resultat
        resultat = c
    return resultat
```

b. Cote finale - retour `float` **(chap. 3, ex. 11).**

```
float coteFinale(float bilan, examen)
    float resultat
    if bilan > examen
        resultat = 0.25*bilan + 0.75*examen
    else
        resultat = examen
    return resultat
```

c. Année bissextile - retour `Boolean` (chap. 3, ex. 10).

```
Boolean estBissextile(int annee)
    Boolean resultat
    resultat = (annee MOD 400 == 0) OR
               ((annee MOD 4 == 0) AND (annee MOD 100 ≠ 0))
    return resultat
```

d. Signe d'un nombre - retour `String` (chap. 3, ex. 4).

```
String nomSigne(float x)
    String resultat
    if x < 0
        resultat = "strictement négatif"
    else if x > 0
        resultat = "strictement positif"
    else
        resultat = "nul"
    return resultat
```

Contrat : ces modules n'effectuent aucun `read` et aucun `print`. Le code appelant peut utiliser le résultat, par exemple `print nomSigne(-4)` ou `print coteFinale(16,12)`.

Exercice 5 - Valeur absolue

Source : p. 42 (PDF 51).

```
float valeurAbsolue(float x)
    float resultat
    if x < 0
        resultat = -x
    else
        resultat = x
    return resultat
```

Contrôles : `valeurAbsolue(-3.5)=3.5`, `valeurAbsolue(0)=0`, `valeurAbsolue(7)=7`. La valeur retournée est toujours ≥ 0.

Exercice 6 - Maximum de 4 nombres

Source : p. 42 (PDF 51). Trois appels au module du maximum de deux entiers : deux comparaisons par paires, puis une comparaison finale.

```
int maximum(int x, y)
    int resultat
    if x > y
        resultat = x
    else
        resultat = y
    return resultat

int maximum4(int a, b, c, d)
    int maxGauche, maxDroite, resultat
    maxGauche = maximum(a, b)
    maxDroite = maximum(c, d)
    resultat = maximum(maxGauche, maxDroite)
    return resultat
```

Contrôles : `(4,2,9,1)` → `9` ; `(-4,-2,-9,-1)` → `-1` ; `(3,3,3,3)` → `3`.

Exercice 7 - Validation de date

Source : p. 42 (PDF 51). Quatre modules, dans l'ordre demandé. On adopte la contrainte **année ≥ 1** (réponse à la réflexion finale) et le calendrier grégorien.

1. Année bissextile.

```
Boolean estBissextile(int annee)
    Boolean resultat
    resultat = (annee MOD 400 == 0) OR
               ((annee MOD 4 == 0) AND (annee MOD 100 ≠ 0))
    return resultat
```

2. Jours du mois. Précondition : mois de 1 à 12 et année ≥ 1.

```
int nombreJours(int mois, annee)
    int jours
    switch mois
        1, 3, 5, 7, 8, 10, 12 : jours = 31
        4, 6, 9, 11 : jours = 30
        2 :
            if estBissextile(annee)
                jours = 29
            else
                jours = 28
    return jours
```

3. Vérification de l'intervalle fermé [1,max].

```
Boolean dansIntervalle(int x, max)
    Boolean resultat
    resultat = x ≥ 1 AND x ≤ max
    return resultat
```

4. Vérification complète de la date. Ne pas appeler `nombreJours` avant d'avoir vérifié la validité du mois.

```
Boolean dateValide(int J, M, A)
    Boolean valide
    valide = false
    if A ≥ 1 AND dansIntervalle(M, 12)
        valide = dansIntervalle(J, nombreJours(M, A))
    return valide
```

Contrôles : 29/02/2024 → `true` ; 29/02/2025 → `false` ; 29/02/1900 → `false` ; 29/02/2000 → `true` ; 31/04/2026 → `false` ; jour 0, mois 13 ou année 0 → `false`. Une entrée hors du domaine des sous-modules doit être rejetée **avant** leur appel.